

Understanding the Master Theorem

A short reading · TC1031 Data Structures and Fundamental Algorithms

Where the shortcut comes from

A recursive algorithm calls itself on a smaller version of the same problem, so its running time can be written as a recurrence relation: an equation for $T(n)$ that refers to T of a smaller input, plus whatever work the function does on its own. You can always solve a recurrence by hand — expanding it level by level until a pattern shows up — but that takes real work every time. The **Master Theorem** is a shortcut: a ready-made answer for one very common shape of recurrence, where an algorithm makes several recursive calls, each on an equal-sized slice of the input, plus some extra work of its own:

$$T(n) = a \cdot T(n / b) + O(n^d)$$

a — how many recursive calls the function makes.

b — how much smaller each call's input gets (a call handles n / b elements).

d — the extra work done at each call, outside the recursive calls themselves, expressed as $O(n^d)$.

The idea: two forces racing

Picture the recursion tree. Two things are competing to dominate the total cost: how fast the number of recursive calls explodes (that grows like $n^{(\log_b a)}$), and how much the extra work at each level adds up to (n^d , repeated over $\log_b n$ levels). The Master Theorem simply asks which one wins, by comparing **a** to **b^d**:

Compare a to b ^d	Result	What it means
$a > b^d$	$O(n^{(\log_b a)})$	the recursive calls dominate
$a = b^d$	$O(n^d \cdot \log n)$	perfectly balanced — merge sort lives here
$a < b^d$	$O(n^d)$	the extra work dominates

A simple example first

Take a function that does nothing but split its work in half, twice, with no real combining step:

```
void visit(int n) {
    if (n <= 1) return;
    visit(n / 2);
    visit(n / 2);
}
```

It makes 2 recursive calls, each on $n/2$ elements, and does only a constant amount of work besides the calls (no loop, no merge — just $O(1)$). Reading a, b and d off that definition:

Step	
Identify	$a = 2, b = 2, d = 0$ (extra work is $O(1) = O(n^0)$)
Compute b ^d	$b^d = 2^0 = 1$

Compare	$a = 2 > b^d = 1$
Case	$a > b^d \rightarrow$ the recursive calls dominate
Result	$O(n^{\log_b a}) = O(n^{\log_2 2}) = O(n^1) = O(n)$

That is the whole mechanic: name a , b and d from the code, compute b^d , compare it to a , and read the matching row off the table above.

Applying it to merge sort

mergeSort follows the same shape, but this time there *is* real extra work: the merge step, which costs $O(n)$.

Step	
Identify	$a = 2$ (mergeSort calls itself twice)
	$b = 2$ (each call handles $n/2$ elements)
	$d = 1$ (merge costs $O(n) = O(n^1)$)
Compute b^d	$b^d = 2^1 = 2$
Compare	$a = 2 = b^d = 2$
Case	$a = b^d \rightarrow$ perfectly balanced, the tie case
Result	$O(n^d \cdot \log n) = O(n^1 \cdot \log n) = O(n \log n)$